

Compte rendu - Lab 4

SQL analytique avec DuckDB

Requêtes sur la couche bronze S3, SQL analytique, Parquet, partitionnement
Hive et API Python, sur la plateforme Onyxia (Adaltas)

Auteure	Dorline TIAKO OKASSIE
Groupe	ING4 – Groupe 01 – Data & Intelligence Artificielle
Environnement	Service Onyxia <code>vscode-pyspark</code> , DuckDB v1.5.5, stockage S3 SeaweedFS
Bucket	<code>s3://user-d-tiakookassie-ece</code>
Dépôt Git	<code>github.com/Dorline0121/ece-2026-bigdata</code> – commit <code>75ae0ab</code>
Date	2 octobre 2026

Sommaire

1. Objectifs du lab
2. Particularités de l'environnement Onyxia et contournements
3. Préparation de l'environnement
4. Configuration S3 et fichier d'initialisation
5. Interrogation de la couche bronze
6. Chargement des tables et contrôles de qualité
7. SQL analytique : agrégations, jointures, fenêtrage, pivot
8. Exercices
9. Export Parquet et partitionnement Hive
10. CSV vs Parquet à grande échelle
11. API Python, commit et nettoyage
12. Réponses aux questions du lab
13. Conclusion

1. Objectifs du lab

- Utiliser le CLI DuckDB fourni par l'image Onyxia.
- Interroger directement sur S3 les fichiers CSV de la couche bronze (`users.csv`, `orders.csv`).
- Écrire des requêtes analytiques : agrégations, jointures, fonctions de fenêtrage, pivot.
- Convertir les données en Parquet et mesurer les gains d'un format colonne.
- Exécuter des requêtes DuckDB depuis un script Python intégré au projet `uv`.

2. Particularités de l'environnement Onyxia et contournements

Quatre particularités ont empêché d'appliquer le sujet tel quel. Chacune a été contournée comme suit.

① **Projet `uv` placé directement dans `~/work`**

Constat. Le sujet suppose un dépôt cloné dans un sous-dossier (`/home/onyxia/work/$GIT_REPO_NAME`). Sur mon service, le dépôt Git est `~/work` lui-même (`pyproject.toml`, `src/` et `uv.lock` y sont à la racine). Le package Python s'appelle donc `work`. Par ailleurs, la valeur d'exemple `<git-repo-name>`, collée telle quelle, provoque une erreur bash : `<` et `>` sont des opérateurs de redirection.

Contournement. Pas de `cd` : tout le lab est exécuté depuis `~/work`, et `<uv_project_name>` est remplacé par `work` (`src/work/orders_report.py`, `work.orders_report:main`).

② Collage multiligne cassé dans le CLI DuckDB du terminal VS Code

Constat. Dans le terminal web de VS Code (Onyxia), coller une requête sur plusieurs lignes dans le prompt DuckDB supprime les retours à la ligne. Les mots se retrouvent collés (`birthdateFROM, false)LIMIT`) et DuckDB renvoie une *Parser Error* (figure 8).

Contournement. Toutes les requêtes SQL ont été réécrites **sur une seule ligne**. En bash, au contraire, le collage multiligne fonctionne : la boucle de mesure et les *heredocs* ont été conservés tels quels.

③ Jeu de données généré différent de celui du sujet

Constat. Les données sont générées à la volée par `uv run dataset-*`. Mes dates de naissance sont décalées d'environ 15 jours par rapport au sujet. Surtout, le gros jeu de données ne compte que **252 416 commandes** (29,6 Mo), avec des dates allant de 2020 à **2048**. Le sujet affiche 999 490 commandes, des dates sur plus d'un siècle et un CSV de 117 Mo. Le filtre du sujet `date >= '2100-01-01'` ne correspondait donc à aucune ligne chez moi : les 3 row groups étaient sautés, et la démonstration ne montrait plus rien.

Contournement. Contrôle préalable de la plage de dates (`min/max`) et des statistiques des row groups, puis remplacement du filtre par `date >= '2045-01-01'`. Cette valeur tombe au milieu des données, ce qui permet d'observer un *row group skipping* partiel.

④ Mesure réseau / parsing : contraintes du conteneur non root

Constat. Pour séparer le temps réseau du temps de parsing, j'ai d'abord téléchargé le CSV vers `/dev/null` avec l'AWS CLI. Dans le conteneur Onyxia, l'utilisateur `onyxia` n'est pas root. La CLI affiche donc un avertissement, car elle ne peut pas modifier la date de `/dev/null`. De plus, le temps mesuré (2,46 s) inclut le démarrage de l'AWS CLI, qui est un programme Python, et ne reflète pas le seul transfert.

Contournement. Comparaison de deux exécutions DuckDB de la même requête : CSV lu sur S3 (1,20 s) contre copie locale `orders_large.csv` (0,24 s). L'écart ($\approx 0,96$ s) correspond au réseau. L'avertissement est sans conséquence : le téléchargement a bien eu lieu.

Autres ajustements mineurs : `*.csv` était déjà présent dans le `.gitignore` (doublet supprimé avec `sed -i 's/d/ '`). Le sujet contient aussi une coquille : `$pworkroduct` doit se lire `$product`.

3. Préparation de l'environnement

Définition des variables du lab S3 (endpoint et bucket), puis vérification du projet, du dépôt Git, de la couche bronze et de DuckDB.

```
export S3_ENDPOINT_URL=$(aws configure get endpoint_url --profile 'default' || echo "https://$AWS_
S3_ENDPOINT")
export LAB_BUCKET_NAME="$KUBERNETES_NAMESPACE"
echo "$S3_ENDPOINT_URL $LAB_BUCKET_NAME"
ls /home/onyxia/work/
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

● onyxia@vscode-pyspark-631287-0:~/work$ GIT_REPO_NAME=<nom-de-ton-repo>
cd /home/onyxia/work/$GIT_REPO_NAME
export S3_ENDPOINT_URL=$(
  aws configure get endpoint_url --profile 'default' \
  || echo "https://$AWS_S3_ENDPOINT"
)
export LAB_BUCKET_NAME="$KUBERNETES_NAMESPACE"
echo "$S3_ENDPOINT_URL $LAB_BUCKET_NAME"
bash: syntax error near unexpected token `newline'
https://s3.seaweedfs.adm.adaltas.cloud user-d-tiakookassie-ece
○ onyxia@vscode-pyspark-631287-0:~/work$
```

Figure 1 - Variables définies (endpoint SeaweedFS, bucket user-d-tiakookassie-ece). L'erreur de syntaxe vient du placeholder <...> collé tel quel (particularité ①)

```
● onyxia@vscode-pyspark-631287-0:~/work$ ls /home/onyxia/work/
job-upload-bronze.yaml orders.csv pyproject.toml README.md src users.csv uv.lock
○ onyxia@vscode-pyspark-631287-0:~/work$
```

Figure 2 - Le projet uv se trouve directement dans ~/work

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

onyxia@vscode-pyspark-631287-0:~/work$ ls /home/onyxia/work/
● onyxia@vscode-pyspark-631287-0:~/work$ ls src/
git status | head -3
aws s3 --profile 'default' ls "s3://$LAB_BUCKET_NAME/bronze/"
duckdb --version
duckdb -c "SELECT name, type, provider, persistent, storage FROM duckdb_secrets();"
work
On branch main
Your branch is up to date with 'origin/main'.

2026-10-02 13:25:33 331568 orders.csv
```

Figure 3 - Package work, branche main à jour, orders.csv présent dans bronze/

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

onyxia@vscode-pyspark-631287-0:~/work$ ls src/
git status | head -3
aws s3 --profile 'default' ls "s3://$LAB_BUCKET_NAME/bronze/"
duckdb --version ...
```

name	type	provider	persistent	storage
s3_onyxia_connection	s3	config	true	local_file

```
onyxia@vscode-pyspark-631287-0:~/work$
```

Figure 4 - Secret persistant s3_onyxia_connection créé par Onyxia (type s3, stockage local_file)

4. Configuration S3 et fichier d'initialisation

Le fichier `init.sql` définit la variable `bucket` et le fuseau UTC. Les fichiers générés sont exclus du dépôt.

```
cat > init.sql <<SQL
SET VARIABLE bucket = 's3://$LAB_BUCKET_NAME';
SET TimeZone = 'UTC';
SQL
printf '*.duckdb\ninit.sql\n' >> .gitignore
```

```
onyxia@vscode-pyspark-631287-0:~/work$ cat > init.sql <<SQL
SET VARIABLE bucket = 's3://$LAB_BUCKET_NAME';
SET TimeZone = 'UTC';
SQL
cat init.sql
SET VARIABLE bucket = 's3://user-d-tiakookassie-ece';
SET TimeZone = 'UTC';
onyxia@vscode-pyspark-631287-0:~/work$
```

Figure 5 - Contenu de `init.sql` après génération

```
onyxia@vscode-pyspark-631287-0:~/work$ sed -i '$d' .gitignore
cat .gitignore
.*
!.gitignore
!.python-version
__pycache__/
*.csv
*.duckdb
init.sql
onyxia@vscode-pyspark-631287-0:~/work$
```

Figure 6 - `.gitignore` final : `*.csv`, `*.duckdb`, `init.sql`

```
onyxia@vscode-pyspark-631287-0:~/work$ duckdb analytics.duckdb -init init.sql
-- Loading resources from init.sql
DuckDB v1.5.5 (Variegata)
Enter ".help" for usage hints.
analytics D SELECT getvariable('bucket') AS bucket;
```

bucket
s3://user-d-tiakookassie-ece

Figure 7 - Ouverture de `analytics.duckdb` avec `-init` : la variable `bucket` est bien définie

```
analytics D SELECT uuid, username, name, birthdate FROM read_csv(getvariable('bucket') || '/bronze/users.csv', header = true, strict_mode = false) LIMIT 3;
Parser Error:
syntax error at or near "("

LINE 1: SELECT uuid, username, name, birthdate FROM read_csv(getvariable('bucket') || '/bronze/users.csv', header =...
                                         ^
analytics D
```

Figure 8 - Parser Error due à la perte des retours à la ligne lors du collage (particularité ②)

5. Interrogation de la couche bronze

`read_csv` lit les fichiers directement sur S3, sans les charger. DuckDB détecte le dialecte et les types par échantillonnage.

```
SELECT uuid, username, name, birthdate FROM read_csv(getvariable('bucket') || '/bronze/users.csv',
header = true, strict_mode = false) LIMIT 3;
```

```
analytics D SELECT uuid, username, name, birthdate FROM read_csv(getvariable('bucket') || '/bronze/users.csv', header = true, strict_mode = false) LIMIT 3;
```

uuid	username	name	birthdate
varchar	varchar	varchar	date
bdd640fb-0667-4ad1-9c80-317fa3b1799d	garzaanthony	Charles Garcia	1935-09-23
17be3111-1a2a-43ed-962b-0f79c37459ee	blairamanda	Ryan Munoz	2009-12-19
47294739-614f-43d7-99db-3ad0ddd1dfb2	elizabethmiles	Jacob Wood	1941-10-22

Figure 9 - Trois premiers utilisateurs lus sur S3 (dates de naissance décalées d'environ 15 jours par rapport au sujet)

```
analytics D DESCRIBE FROM read_csv(getvariable('bucket') || '/bronze/orders.csv');
```

Describe	
uuid	varchar
user_uuid	varchar
date	timestamp with time zone
quantity	bigint
product	varchar

Figure 10 - Schéma inféré de orders.csv

column_name	column_type	approx_unique	null_percentage
varchar	varchar	int64	decimal(9,2)
uuid	VARCHAR	3167	0.00
user_uuid	VARCHAR	40	0.00
date	TIMESTAMP WITH TIME ZONE	3566	0.00
quantity	BIGINT	5	0.00
product	VARCHAR	6	0.00

Figure 11 - SUMMARIZE : approx_unique = 40 pour user_uuid et 3167 pour uuid, aucune valeur nulle

6. Chargement des tables et contrôles de qualité

```
CREATE OR REPLACE TABLE users AS FROM read_csv(getvariable('bucket') || '/bronze/users.csv');
CREATE OR REPLACE TABLE orders AS FROM read_csv(getvariable('bucket') || '/bronze/orders.csv');
```

```
analytics D CREATE OR REPLACE TABLE users AS FROM read_csv(getvariable('bucket') || '/bronze/users.csv');
analytics D CREATE OR REPLACE TABLE orders AS FROM read_csv(getvariable('bucket') || '/bronze/orders.csv');
analytics D .tables
```

analytics

main

users

uuid	varchar
username	varchar
name	varchar
sex	varchar
address	varchar
mail	varchar
birthdate	date

50 rows

orders

uuid	varchar
user_uuid	varchar
date	timestamp with time zone
quantity	bigint
product	varchar

2829 rows

Figure 12 - .tables : users (50 lignes, 7 colonnes) et orders (2829 lignes, 5 colonnes)

```
analytics D SELECT min(date) AS first_order, max(date) AS last_order FROM orders;
```

first_order	last_order
timestamp with time zone	timestamp with time zone
2020-01-01 00:02:17.891488+00	2020-04-27 20:12:06.167622+00

Figure 13 - Période couverte : du 2020-01-01 au 2020-04-27

```
analytics D SELECT count(*) AS orphan_orders FROM orders o ANTI JOIN users u ON o.user_uuid = u.uuid;
```

orphan_orders
int64
0

Figure 14 - Commandes orphelines : 0

```
analytics D SELECT count(*) AS inactive_users FROM users u ANTI JOIN orders o ON o.user_uuid = u.uuid;
```

inactive_users
int64
0

Figure 15 - Utilisateurs sans commande : 0

```
analytics D SELECT uuid, count(*) AS occurrences FROM orders GROUP BY uuid HAVING count(*) > 1;
```

uuid	occurrences
varchar	int64
0 rows	

Figure 16 - Identifiants de commande dupliqués : aucun

Résultat. Les trois hypothèses sont vérifiées : intégrité référentielle, aucun utilisateur inactif, unicité des commandes.

7. SQL analytique

7.1 Agrégations

```
analytics D SELECT product, count(*) AS orders, sum(quantity) AS quantity, round(100 * sum(quantity) / sum(sum(quantity)) OVER (), 1) AS share_pct FROM orders GROUP BY product ORDER BY quantity DESC;
```

product	orders	quantity	share_pct
varchar	int64	int128	double
croissant	491	1523	17.7
drink	484	1493	17.3
brioche	468	1424	16.5
bread	458	1420	16.5
donut	462	1390	16.1
cookie	466	1365	15.8

Figure 17 - Quantité vendue par produit et part du total (sum(sum()) OVER ())

```
analytics D SELECT date_trunc('month', date) AS month, count(*) AS orders, sum(quantity) AS quantity FROM orders GROUP BY month ORDER BY month;
```

month timestamp with time zone	orders int64	quantity int128
2020-01-01 00:00:00+00	744	2297
2020-02-01 00:00:00+00	696	2154
2020-03-01 00:00:00+00	744	2192
2020-04-01 00:00:00+00	645	1972

Figure 18 – Commandes par mois : 744 / 696 / 744 / 645, soit une commande par heure

7.2 Jointures

```
analytics D SELECT u.username, u.name, count(*) AS orders, sum(o.quantity) AS quantity FROM orders o JOIN users u ON o.user_uuid = u.uuid GROUP BY ALL ORDER BY quantity DESC LIMIT 5;
```

username varchar	name varchar	orders int64	quantity int128
elizabeth57	Connie Holt	97	317
karenhudson	Angelica Keith	97	303
jsmith	Shane Alexander	100	301
hoganashlee	Patricia Jefferson	100	289
wrightjames	Monica McBride	92	285

Figure 19 – Top 5 des clients (fait orders × dimension users)

Au 5^e rang, j'obtiens `wrightjames` là où le sujet affiche `stephen00`. Les deux clients sont **ex æquo** à **285**, et `ORDER BY quantity DESC` ne départage pas les égalités. Pour un résultat déterministe, il faut ajouter un critère secondaire : `ORDER BY quantity DESC, username`.

```
analytics D SELECT (date_diff('year', u.birthdate, DATE '2020-01-01') // 20) * 20 AS age_group, count(DISTINCT u.uuid) AS users, count(*) AS orders, round(avg(o.quantity), 2) AS avg_quantity FROM orders o JOIN users u ON o.user_uuid = u.uuid GROUP BY age_group ORDER BY age_group;
```

age_group int64	users int64	orders int64	avg_quantity double
0	14	686	3.08
20	7	483	3.02
40	12	721	3.02
60	11	694	3.02
80	5	198	3.14
100	1	47	3.19

Figure 20 – Commandes par tranche d'âge de 20 ans

7.3 Fonctions de fenêtrage

```
analytics D WITH daily AS (SELECT date::DATE AS day, sum(quantity) AS quantity FROM orders GROUP BY day) SELECT day, quantity, sum(quantity) OVER (ORDER BY day) AS cumulative, round(avg(quantity) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW), 1) AS avg_7d FROM daily ORDER BY day LIMIT 10;
```

day date	quantity int128	cumulative int128	avg_7d double
2020-01-01	86	86	86.0
2020-01-02	62	148	74.0
2020-01-03	80	228	76.0
2020-01-04	66	294	73.5
2020-01-05	72	366	73.2
2020-01-06	75	441	73.5
2020-01-07	73	514	73.4
2020-01-08	81	595	72.7
2020-01-09	89	684	76.6
2020-01-10	75	759	75.9

10 rows

4 columns

Figure 21 – Quantité par jour, cumul et moyenne mobile sur 7 jours

10 rows • 4 columns

```
analytics D SELECT strftime(date, '%Y-%m') AS month, product, sum(quantity) AS quantity FROM orders GROUP BY month, product QUALIFY rank() OVER (PARTITION BY month ORDER BY sum(quantity) DESC) = 1 ORDER BY month;
```

month	product	quantity
2020-01	donut	438
2020-02	croissant	392
2020-03	bread	411
2020-04	croissant	386

Figure 22 – Produit le plus vendu de chaque mois (rank() + QUALIFY)

7.4 Pivot

```
analytics D PIVOT (SELECT strftime(date, '%Y-%m') AS month, product, quantity FROM orders) ON product USING sum(quantity) ORDER BY month;
```

month	bread	brioche	cookie	croissant	donut	drink
2020-01	315	431	376	360	438	377
2020-02	362	350	371	392	314	365
2020-03	411	333	340	385	321	402
2020-04	332	310	278	386	317	349

Figure 23 – Tableau croisé mois × produit

8. Exercices

Exercice 1 – Commandes par utilisateur et quantité par commande

```
SELECT round(count(*) / count(DISTINCT user_uuid), 2) AS avg_orders_per_user, round(avg(quantity), 2) AS avg_quantity_per_order FROM orders;
```

```
analytics D SELECT round(count(*) / count(DISTINCT user_uuid), 2) AS avg_orders_per_user, round(avg(quantity), 2) AS avg_quantity_per_order FROM orders;
```

avg_orders_per_user	avg_quantity_per_order
56.58	3.05

Figure 24 – En moyenne, 56,58 commandes par utilisateur et 3,05 articles par commande

Exercice 2 – Première et dernière commande de chaque utilisateur

```
SELECT u.username, min(o.date) AS first_order, max(o.date) AS last_order, date_diff('day', min(o.date), max(o.date)) AS days_between FROM orders o JOIN users u ON o.user_uuid = u.uuid GROUP BY ALL ORDER BY days_between DESC;
```

```
analytics D SELECT u.username, min(o.date) AS first_order, max(o.date) AS last_order, date_diff('day', min(o.date), max(o.date)) AS days_between
n FROM orders o JOIN users u ON o.user_uuid = u.uuid GROUP BY ALL ORDER BY days_between DESC;
```

username varchar	first_order timestamp with time zone	last_order timestamp with time zone	days_between int64
hoganashlee	2020-02-06 22:28:23.377915+00	2020-02-11 01:57:32.773432+00	5
jsmith	2020-03-13 06:21:16.892555+00	2020-03-17 09:02:08.476647+00	4
ahancock	2020-03-02 22:15:57.890701+00	2020-03-06 11:03:10.312652+00	4
yobrien	2020-02-22 14:30:47.467747+00	2020-02-26 08:13:22.890319+00	4
wrightjames	2020-01-08 11:20:45.094193+00	2020-01-12 06:23:02.514874+00	4
elizabeth57	2020-02-15 03:39:36.282921+00	2020-02-19 03:23:18.610446+00	4
karenhudson	2020-02-11 02:16:02.488268+00	2020-02-15 02:18:44.854831+00	4
snguyen	2020-01-24 22:10:14.040422+00	2020-01-28 02:03:49.652638+00	4
gilbertwilliam	2020-04-13 21:57:20.638235+00	2020-04-17 07:19:12.128412+00	4
trose	2020-04-08 18:29:41.913897+00	2020-04-12 12:41:22.197741+00	4
stephen00	2020-03-27 11:09:51.257559+00	2020-03-31 06:01:03.068375+00	4
pvalentine	2020-03-06 12:42:24.432345+00	2020-03-10 14:46:03.127489+00	4
mcmillandennis	2020-01-15 18:26:47.867763+00	2020-01-19 06:54:04.981594+00	4
calebrussell	2020-01-31 13:14:46.485026+00	2020-02-03 11:51:33.868588+00	3
mccormickdeborah	2020-04-17 08:11:17.502205+00	2020-04-20 03:47:48.886749+00	3
ronaldstephens	2020-02-26 09:46:06.054277+00	2020-02-29 15:12:25.629535+00	3
danderson	2020-01-28 09:52:38.257356+00	2020-01-31 12:23:43.147693+00	3
farrelldebra	2020-03-31 07:43:54.855528+00	2020-04-03 15:25:47.120338+00	3
cgrimes	2020-03-10 15:24:50.199645+00	2020-03-13 05:28:53.850918+00	3
danamullins	2020-02-19 04:07:14.167143+00	2020-02-22 13:08:39.646253+00	3
.	.	.	.
.	.	.	.
.	.	.	.

Figure 25 - Écart les plus longs (5 jours au maximum)

.	.	.	.
.	.	.	.
.	.	.	.
wilsonsara	2020-01-22 05:16:44.804087+00	2020-01-24 21:56:50.113927+00	2
nicolejohnson	2020-01-20 04:25:18.263357+00	2020-01-22 04:04:28.088379+00	2
sarah35	2020-01-13 00:10:01.066635+00	2020-01-15 17:53:07.152912+00	2
dayderek	2020-02-03 12:36:43.393482+00	2020-02-05 15:47:14.376374+00	2
howelljohn	2020-02-05 16:44:21.721362+00	2020-02-06 21:35:51.491468+00	1
ithomas	2020-01-06 02:48:59.231613+00	2020-01-07 16:59:25.164747+00	1
perezrebecca	2020-01-07 23:37:11.211033+00	2020-01-08 10:45:20.193352+00	1
lozanojulie	2020-03-19 15:32:48.189291+00	2020-03-20 13:07:59.57991+00	1
robin74	2020-03-20 14:40:43.856477+00	2020-03-21 14:06:56.099518+00	1
atkinsbarbara	2020-04-12 13:29:18.732499+00	2020-04-13 02:31:42.803768+00	1
vanessafernandez	2020-03-26 06:02:01.60085+00	2020-03-27 10:12:38.221675+00	1
elizabethmiles	2020-01-05 04:50:10.331513+00	2020-01-06 01:03:27.852411+00	1
hhoffman	2020-01-19 07:24:12.20992+00	2020-01-20 03:24:02.799052+00	1
blairamanda	2020-01-04 11:59:53.242326+00	2020-01-05 03:02:14.273688+00	1
smoore	2020-01-12 07:14:53.401298+00	2020-01-12 23:09:49.797471+00	0
clarence34	2020-01-28 03:45:53.98694+00	2020-01-28 08:41:51.442357+00	0
jessica23	2020-04-13 03:06:40.916881+00	2020-04-13 20:08:28.793364+00	0
kayla51	2020-01-07 17:49:04.785279+00	2020-01-07 20:31:55.964946+00	0
millertodd	2020-01-07 21:58:38.925654+00	2020-01-07 22:48:22.203716+00	0
heatherberger	2020-03-17 10:52:57.361043+00	2020-03-17 17:10:14.610404+00	0

50 rows (40 shown) use .last to show entire result 4 columns

```
analytics D
```

Figure 26 - Écart les plus courts (0 jour)

Analyse. L'écart ne dépasse jamais 5 jours, alors que le jeu de données couvre 4 mois. Le générateur attribue à chaque utilisateur un **bloc de commandes horaires consécutives** : 100 commandes correspondent à environ 4 jours. C'est une limite du jeu de données synthétique.

Exercice 3 - Heure de la journée la plus vendeuse

```
SELECT hour(date) AS hour_of_day, count(*) AS orders, sum(quantity) AS quantity FROM orders GROUP BY hour_of_day ORDER BY quantity DESC LIMIT 1;
```

```
analytics D SELECT hour(date) AS hour_of_day, count(*) AS orders, sum(quantity) AS quantity FROM orders GROUP BY hour_of_day ORDER BY quantity DESC LIMIT 1;
```

hour_of_day int64	orders int64	quantity int128
6	118	385

Figure 27 – Pic à 6 h UTC : 385 articles sur 118 commandes

Analyse. Avec une commande par heure, chaque heure reçoit environ 118 commandes (2829 / 24). Le « pic » ne vient que de quantités aléatoires plus élevées et n'a pas de signification métier.

Exercice 4 – Variation mensuelle par produit (lag)

```
WITH monthly AS (SELECT product, strftime(date, '%Y-%m') AS month, sum(quantity) AS quantity FROM orders GROUP BY ALL) SELECT product, month, quantity, lag(quantity) OVER (PARTITION BY product ORDER BY month) AS prev_quantity, round(100.0 * (quantity - lag(quantity) OVER (PARTITION BY product ORDER BY month)) / lag(quantity) OVER (PARTITION BY product ORDER BY month), 1) AS variation_pct FROM monthly ORDER BY product, month;
```

```
analytics D WITH monthly AS (SELECT product, strftime(date, '%Y-%m') AS month, sum(quantity) AS quantity FROM orders GROUP BY ALL) SELECT product, month, quantity, lag(quantity) OVER (PARTITION BY product ORDER BY month) AS prev_quantity, round(100.0 * (quantity - lag(quantity) OVER (PARTITION BY product ORDER BY month)) / lag(quantity) OVER (PARTITION BY product ORDER BY month), 1) AS variation_pct FROM monthly ORDER BY product, month;
```

product varchar	month varchar	quantity int128	prev_quantity int128	variation_pct double
bread	2020-01	315	NULL	NULL
bread	2020-02	362	315	14.9
bread	2020-03	411	362	13.5
bread	2020-04	332	411	-19.2
brioche	2020-01	431	NULL	NULL
brioche	2020-02	350	431	-18.8
brioche	2020-03	333	350	-4.9
brioche	2020-04	310	333	-6.9
cookie	2020-01	376	NULL	NULL
cookie	2020-02	371	376	-1.3
cookie	2020-03	340	371	-8.4
cookie	2020-04	278	340	-18.2
croissant	2020-01	360	NULL	NULL
croissant	2020-02	392	360	8.9
croissant	2020-03	385	392	-1.8
croissant	2020-04	386	385	0.3
donut	2020-01	438	NULL	NULL
donut	2020-02	314	438	-28.3
donut	2020-03	321	314	2.2
donut	2020-04	317	321	-1.2
drink	2020-01	377	NULL	NULL
drink	2020-02	365	377	-3.2
drink	2020-03	402	365	10.1
drink	2020-04	349	402	-13.2

24 rows 5 columns

Figure 28 – Variations de -28,3 % (donut en février) à +14,9 % (bread en février)

Analyse. La baisse générale d'avril est en partie un artefact : le mois s'arrête le 27. Pour une comparaison juste, il faudrait rapporter les quantités au nombre de jours (moyenne journalière).

Exercice 5 – Utilisateurs ayant commandé tous les produits

```
SELECT u.username, count(DISTINCT o.product) AS nb_products FROM orders o JOIN users u ON o.user_uid = u.uuid GROUP BY ALL HAVING count(DISTINCT o.product) = (SELECT count(DISTINCT product) FROM orders) ORDER BY u.username;
```

karenhudson	6
lozanojulie	6
lucasrodney	6
mariahdavis	6
mccormickdeborah	6
mcmillandennis	6
michael22	6
nicolejohnson	6
pvalentine	6
robin74	6
ronaldstephens	6
sarah35	6
smoore	6
snguyen	6
stephen00	6
trose	6
vanessafernandez	6
wilsonlara	6
wrightjames	6
yobrien	6

45 rows (40 shown) 2 columns

Figure 29 – 45 utilisateurs sur 50 ont commandé les 6 produits

analytics D **SELECT** u.username, count(*) **AS** orders, count(**DISTINCT** o.product) **AS** nb_products **FROM** orders o **JOIN** users u **ON** o.user_uuid = u.uuid **GROUP BY ALL HAVING** count(**DISTINCT** o.product) < 6 **ORDER BY** orders;

username varchar	orders int64	nb_products int64
millertodd	2	2
kayla51	4	2
clarence34	6	4
heatherberger	8	4
perezrebecca	12	5

Figure 30 – Les 5 utilisateurs restants sont les plus petits clients (2 à 12 commandes)

9. Export Parquet et partitionnement Hive

```
COPY orders TO (getvariable('bucket') || '/analytics/orders.parquet') (FORMAT parquet);
```

analytics D **COPY** orders **TO** (getvariable('bucket') || '/analytics/orders.parquet') (FORMAT parquet);
analytics D **SELECT** path_in_schema, type, compression, total_compressed_size, total_uncompressed_size **FROM** parquet_metadata(getvariable('bucket') || '/analytics/orders.parquet');

path_in_schema varchar	type varchar	compression varchar	total_compressed_size int64	total_uncompressed_size int64
uuid	BYTE_ARRAY	SNAPPY	102191	113189
user_uuid	BYTE_ARRAY	SNAPPY	2125	2348
date	INT64	SNAPPY	22667	22661
quantity	INT64	SNAPPY	1242	1245
product	BYTE_ARRAY	SNAPPY	1271	1266

Figure 31 – Métadonnées du footer Parquet : uuid ≈ 102 Ko, user_uuid ≈ 2 Ko (dictionnaire), compression SNAPPY

```
COPY orders TO (getvariable('bucket') || '/analytics/orders_by_product') (FORMAT parquet, PARTITION BY (product));
```

```
analytics D COPY orders TO (getvariable('bucket') || '/analytics/orders_by_product') (FORMAT parquet, PARTITION_BY (product));
analytics D SELECT file FROM glob(getvariable('bucket') || '/analytics/orders_by_product/**');
```

file	varchar
s3://user-d-tiakookassie-ece/analytics/orders_by_product/product=bread/data_0.parquet	
s3://user-d-tiakookassie-ece/analytics/orders_by_product/product=brioche/data_0.parquet	
s3://user-d-tiakookassie-ece/analytics/orders_by_product/product=cookie/data_0.parquet	
s3://user-d-tiakookassie-ece/analytics/orders_by_product/product=croissant/data_0.parquet	
s3://user-d-tiakookassie-ece/analytics/orders_by_product/product=donut/data_0.parquet	
s3://user-d-tiakookassie-ece/analytics/orders_by_product/product=drink/data_0.parquet	

Figure 32 – Un dossier par produit, selon la convention Hive `product=<valeur>`

```
analytics D EXPLAIN ANALYZE SELECT count(*) FROM read_parquet(getvariable('bucket') || '/analytics/orders_by_product/**/*.parquet') WHERE product = 'cookie';
```

Query Profiling Information
EXPLAIN ANALYZE SELECT count(*) FROM read_parquet(getvariable('bucket') '/analytics/orders_by_product/**/*.parquet') WHERE product = 'cookie';
HTTP Stats
In: 33.7 Kio Out: 0 bytes READ: 0 WRITE: 3 REQUEST: 0 RESPONSE: 0 DELETE: 0
Total Time: 0.211s

Figure 33 – EXPLAIN ANALYZE : 3 GET, 33,7 Kio reçus

UNGROUPED_AGGREGATE
Aggregates: count_star()
1 row 0.00s
TABLE_SCAN
Function: READ_PARQUET
File Filters: (product = 'cookie')
Scanning Files: 1/6 Total Files Read: 1
Filename(s): s3://user-d-tiakookassie-ece/analytics/orders_by_product/product=cookie/data_0.parquet
466 rows 0.00s

Figure 34 – TABLE_SCAN : Scanning Files 1/6, 466 lignes

Résultat. Grâce à l'élagage de partitions (*partition pruning*), seul le fichier `product=cookie` est lu.

10. CSV vs Parquet à grande échelle

```
uv run dataset-orders -u 5000 -o csv > orders_large.csv
s5cmd --profile 'default' cp orders_large.csv "s3://$LAB_BUCKET_NAME/large/orders.csv"
```

```
onyxia@vscode-pyspark-631287-0:~/work$ uv run dataset-orders -u 5000 -o csv > orders_large.csv
ls -lh orders_large.csv
s5cmd --profile 'default' cp orders_large.csv "s3://$LAB_BUCKET_NAME/large/orders.csv"
-rw-r--r-- 1 onyxia users 29M Oct 2 14:15 orders_large.csv
cp orders_large.csv s3://user-d-tiakookassie-ece/large/orders.csv
```

Figure 35 – Génération du gros jeu de données (29 Mo) et envoi sur S3


```

|| Total Time: 0.348s ||
● onyxia@vscode-pyspark-631287-0:~/work$ time aws s3 --profile 'default' cp "s3://$LAB_BUCKET_NAME/large/orders.csv" /dev/null
duckdb -c ".timer on" -c "FROM read_csv('orders_large.csv') SELECT product, sum(quantity) GROUP BY product;"
warning: Skipping file /dev/null. Successfully Downloaded /dev/null but was unable to update the last modified time. The file was downloaded, b
ut attempting to modify the utime of the file failed. Is the file owned by another user?
download: s3://user-d-tiakookassie-ece/large/orders.csv to ../../dev/null

real    0m2.464s
user    0m2.035s
sys     0m0.527s
-- Loading resources from /home/onyxia/.duckdbrc

```

product varchar	sum(quantity) int128
brioche	126443
drink	126104
croissant	125715
donut	125979
cookie	125734
bread	127341

```

Run Time (s): real 0.242 user 0.271739 sys 0.045981

```

Figure 39 – Séparation réseau / parsing : la même requête sur la copie locale prend 0,24 s (particularité ④)

11. API Python, commit et nettoyage

```

uv add duckdb
# script src/work/orders_report.py (requête paramétrée par $product)
sed -i '/^dataset-orders = /a orders-report = "work.orders_report:main"' pyproject.toml
uv run orders-report
uv run orders-report -p cookie

```

```

● onyxia@vscode-pyspark-631287-0:~/work$ uv add duckdb
Resolved 4 packages in 313ms
Built work @ file:///home/onyxia/work
Prepared 2 packages in 727ms
Uninstalled 1 package in 0.84ms
Installed 2 packages in 10ms
+ duckdb==1.5.6
~ work==0.1.0 (from file:///home/onyxia/work)

```

Figure 40 – Ajout de la dépendance duckdb 1.5.6

```

● onyxia@vscode-pyspark-631287-0:~/work$ cat <<'PY' >src/work/orders_report.py
import argparse
import os

import duckdb

def orders_report(product=None):
    con = duckdb.connect()
    con.execute("SET TimeZone = 'UTC'")
    bronze = f"s3://{os.environ['LAB_BUCKET_NAME']}/bronze"
    return con.execute(
        f"""
        SELECT strftime(date, '%Y-%m') AS month, count(*) AS orders, sum(quantity) AS quantity
        FROM read_csv('{bronze}/orders.csv', strict_mode = false)
        WHERE $product IS NULL OR product = $product
        GROUP BY month
        ORDER BY month
        """,
        {"product": product},
    ).fetchall()

def main():
    parser = argparse.ArgumentParser(prog="orders-report", description="Monthly orders report")
    parser.add_argument("-p", "--product", help="Filter the orders on a product.")
    args = parser.parse_args()
    for month, orders, quantity in orders_report(args.product):
        print(f"{month}\t{orders}\t{quantity}")

if __name__ == "__main__":
    main()
PY

```

Figure 41 – Création du script orders_report.py

```

● onyxia@vscode-pyspark-631287-0:~/work$ sed -i '/^dataset-orders = /a orders-report = "work.orders_report:main"' pyproject.toml
grep -A4 'project.scripts' pyproject.toml
[project.scripts]
dataset-users = "work.dataset_users:main"
dataset-orders = "work.dataset_orders:main"
orders-report = "work.orders_report:main"

```

Figure 42 – Déclaration de la commande orders-report dans [project.scripts]

```

● onyxia@vscode-pyspark-631287-0:~/work$ uv run orders-report
uv run orders-report -p cookie
Built work @ file:///home/onyxia/work
Uninstalled 1 package in 1ms
Installed 1 package in 8ms
2020-01 744      2297
2020-02 696      2154
2020-03 744      2192
2020-04 645      1972
2020-01 128       376
2020-02 121       371
2020-03 122       340
2020-04 95        278

```

Figure 43 – Rapport mensuel sans filtre, puis filtré sur cookie : résultats identiques au sujet

Le produit est transmis comme **paramètre de requête** (`$product`), et non concaténé dans la chaîne SQL. Cela protège contre l'injection SQL.

```

● onyxia@vscode-pyspark-631287-0:~/work$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitignore
        modified:   pyproject.toml
        modified:   uv.lock

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        src/work/orders_report.py

no changes added to commit (use "git add" and/or "git commit -a")

```

Figure 44 – git status : analytics.duckdb, init.sql et orders_large.csv sont bien ignorés

```

no changes added to commit (use "git add" and/or "git commit -a")
● onyxia@vscode-pyspark-631287-0:~/work$ git add .gitignore pyproject.toml uv.lock src
git commit -m "feat: orders report with duckdb"
git push
[main 75ae0ab] feat: orders report with duckdb
 4 files changed, 63 insertions(+), 1 deletion(-)
 create mode 100644 src/work/orders_report.py
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 16 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (8/8), 3.58 KiB | 1.19 MiB/s, done.
Total 8 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Dorline0121/ece-2026-bigdata.git
 134d464..75ae0ab main -> main

```

Figure 45 – Commit 75ae0ab « feat: orders report with duckdb » poussé sur GitHub

```

● onyxia@vscode-pyspark-631287-0:~/work$ aws s3 --profile 'default' rm "s3://$LAB_BUCKET_NAME/analytics/" --recursive
aws s3 --profile 'default' rm "s3://$LAB_BUCKET_NAME/large/" --recursive
rm -f orders_large.csv
aws s3 --profile 'default' ls "s3://$LAB_BUCKET_NAME/" --recursive
delete: s3://user-d-tiakookassie-ece/analytics/orders_by_product/product=bread/data_0.parquet
delete: s3://user-d-tiakookassie-ece/analytics/orders_by_product/product=croissant/data_0.parquet
delete: s3://user-d-tiakookassie-ece/analytics/orders.parquet
delete: s3://user-d-tiakookassie-ece/analytics/orders_by_product/product=brioche/data_0.parquet
delete: s3://user-d-tiakookassie-ece/analytics/orders_by_product/product=donut/data_0.parquet
delete: s3://user-d-tiakookassie-ece/analytics/orders_by_product/product=drink/data_0.parquet
delete: s3://user-d-tiakookassie-ece/analytics/orders_by_product/product=cookie/data_0.parquet
delete: s3://user-d-tiakookassie-ece/large/orders.parquet
delete: s3://user-d-tiakookassie-ece/large/orders.csv
2026-10-02 13:25:33      331568 bronze/orders.csv
2026-10-02 13:25:28       7351 bronze/users.csv

```

Figure 46 – Nettoyage : seuls bronze/orders.csv et bronze/users.csv sont conservés

12. Réponses aux questions du lab

12.1 Configuration S3

Q1. Le secret persistant est stocké dans un fichier du répertoire personnel. Quels sont les risques, et pourquoi sont-ils limités sur Onyxia ?

Les identifiants S3 sont écrits sur disque dans `~/ .duckdb/stored_secrets`. Tout processus lancé sous mon utilisateur peut donc les lire, et ils peuvent fuiter par un commit Git, une sauvegarde ou une archive du répertoire. Sur Onyxia, ce risque est limité pour trois raisons : les identifiants sont **temporaires** (STS, avec un *session token* qui expire, d'où les erreurs `ExpiredToken/403`) ; le conteneur est **personnel et isolé** (un pod Kubernetes par utilisateur) ; les droits sont restreints à **mon seul bucket**.

Q2. Le secret est visible par tout processus de l'utilisateur. Comment restreindre l'accès S3 pour un Job Kubernetes ?

On n'utilise pas ses identifiants personnels. On crée une **identité dédiée** au Job (ServiceAccount ou compte S3 technique), associée à une **politique de moindre privilège** : seulement `GetObject/PutObject`, et seulement sur le préfixe utile (par exemple `bronze/`). Les identifiants sont injectés par un **Secret Kubernetes** monté uniquement dans ce pod. L'idéal est d'utiliser des identifiants courts, obtenus par fédération (OIDC / *AssumeRoleWithWebIdentity*), plutôt que des clés permanentes.

12.2 Couche bronze

Q3. Le jeu de données contient 50 utilisateurs et 2829 commandes uniques. Pourquoi `approx_unique` renvoie-t-il 40 et 3167 ?

`SUMMARIZE` ne compte pas les valeurs distinctes de façon exacte. Il utilise un **estimateur probabiliste HyperLogLog**, rapide et peu gourmand en mémoire mais entaché d'une marge d'erreur. Sur de petites cardinalités, l'écart relatif peut atteindre 10 à 20 %. Le compte exact s'obtient avec `count(DISTINCT ...)`, qui est plus coûteux.

Q4. Les types sont inférés sur un échantillon. Quel problème peut survenir avec un gros fichier dont les premières lignes ne sont pas représentatives ?

Une colonne peut recevoir un type trop restrictif. Par exemple, elle sera typée `BIGINT` si l'échantillon ne contient que des entiers, alors que des décimales ou du texte apparaissent plus loin. La lecture échoue alors avec une erreur de conversion, ou rejette les lignes concernées (`strict_mode = false`, `ignore_errors`). On l'évite en lisant tout le fichier pour l'inférence (`sample_size = -1`), en déclarant explicitement les types (`types / columns`), ou en utilisant un format typé comme Parquet.

12.3 Export Parquet

Q5. Pourquoi la colonne `uuid` est-elle à peine compressée ?

Chaque `uuid` est unique et aléatoire, donc à forte entropie. Aucun encodage ne s'applique : pas de dictionnaire (aucune répétition), pas de run-length (aucune suite identique), pas de delta

(aucune valeur proche de la précédente). Snappy ne trouve pas non plus de motifs répétés. À l'inverse, `user_uuid` n'a que 50 valeurs : grâce au dictionnaire, ses 2829 identifiants tiennent en 2 Ko.

Q6. Pourquoi la taille compressée de certaines colonnes est-elle supérieure à leur taille non compressée ?

La taille « non compressée » est mesurée **après encodage** Parquet (dictionnaire pour `product`, binaire INT64 pour `date`). Les données sont donc déjà compactes, ou quasi aléatoires (microsecondes des timestamps). Snappy n'arrive plus à les réduire, mais ajoute ses propres en-têtes : `date` passe de 22 661 à 22 667 octets et `product` de 1266 à 1271.

12.4 Partitionnement Hive

Q7. Pourquoi partitionner par `uuid` est-il une mauvaise idée ?

On obtiendrait un dossier et un fichier **par commande** : 2829 fichiers minuscules ici, des millions en production. C'est le problème des petits fichiers (*small files problem*), aggravé par le stockage objet : chaque objet coûte au moins une requête HTTP à forte latence, les requêtes sont facturées, le `LIST` est paginé par 1000 clés, les « dossiers » ne sont que des préfixes, et chaque fichier Parquet transporte son propre footer, qui finit par peser plus que les données. L'élagage n'apporte rien non plus, puisqu'on filtre rarement sur un identifiant précis.

Q8. Quelle colonne de partitionnement choisir pour des commandes qui arrivent chaque jour ?

La **date de commande**, tronquée au jour (`order_date=2026-10-02`) ou au mois (`year=/month=`) selon le volume. Les requêtes analytiques filtrent presque toujours sur une période. Les nouvelles données arrivent dans une nouvelle partition, sans réécrire les anciennes. Enfin, le nombre de partitions et leur taille restent maîtrisés : on vise des fichiers d'environ 100 Mo ou plus.

12.5 CSV vs Parquet à grande échelle

Q9. Combien de row groups contient le fichier, et combien sont sautés par le filtre ?

Le fichier contient **3 row groups** : 123 573, 124 276 et 4 567 lignes. Avec le filtre `date >= '2045-01-01'`, **1 row group sur 3 est sauté** : le groupe 0, dont le maximum est le 2034-02-04. Les groupes 1 (2034–2048) et 2 (2048) peuvent contenir des lignes qui correspondent et sont lus. Avec le filtre du sujet (`>= 2100`), les 3 groupes seraient sautés sur mon jeu de données, dont la date maximale est le 2048-10-17.

Q10. Les commandes sont triées par date. Que deviendrait l'efficacité du filtre si elles étaient mélangées ?

Chaque row group contiendrait des dates réparties sur toute la période. Ses statistiques min/max couvriraient donc la plage complète (2020–2048), et **aucun row group ne pourrait être sauté** : il faudrait lire toute la colonne `date`. Le *predicate pushdown* n'est efficace que si les données sont triées ou regroupées selon la colonne filtrée. On trie donc avant d'écrire (`COPY (FROM t ORDER BY date) TO ...`) ou on partitionne par date.

Q11. Comparez les temps CSV et Parquet. Quelle part de l'écart vient du réseau, quelle part du parsing du CSV ?

La requête CSV prend 1,20 s depuis S3 et 0,24 s sur la copie locale. Le **parsing** (découpage du texte, gestion des guillemets, conversion des 252 416 timestamps sur toutes les colonnes) représente donc environ **20 %** du temps. Le **réseau** (28,2 Mio à transférer) en représente environ **80 %**. La requête Parquet (0,34 s) ne transfère que 203 Kio sur 2 colonnes binaires : son temps vient surtout de la **latence** des 4 allers-retours HTTP (footer, puis blocs de colonnes), et non du débit. Sur un réseau plus lent ou plus chargé, l'écart entre CSV et Parquet se creuserait encore.

13. Conclusion

Ce lab montre qu'un moteur embarqué comme DuckDB suffit à mener une analyse complète directement sur le stockage objet, sans serveur, de la lecture des CSV bruts jusqu'à un script Python intégré au projet. Les mesures confirment les apports d'un format colonne : Parquet est 2,6 fois plus compact et lit environ 140 fois moins de données pour une agrégation. Ses statistiques (row groups) et le partitionnement Hive permettent en plus de sauter les données inutiles. Ces gains dépendent toutefois de l'organisation physique des données : tri, choix de la colonne de partition, taille des fichiers. Enfin, les écarts observés avec le sujet (jeu de données généré, égalités de classement, mois incomplet) rappellent qu'il faut toujours vérifier ses données avant d'interpréter un résultat, ce que la couche silver et les tests dbt du prochain module systématiseront.